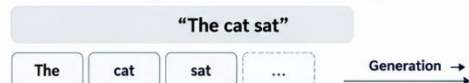


KV Cache

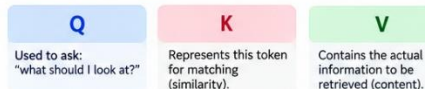
1 The Setting: Generating a Sentence

We want the model to generate the next tokens for the prompt:



2 What are K and V?

For each token, the model creates three vectors:

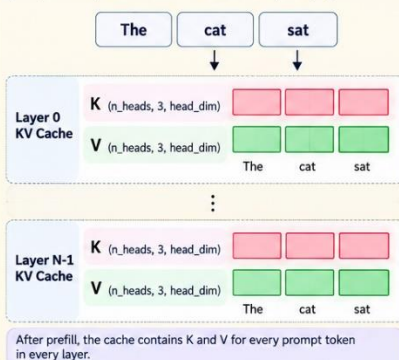


At each layer, for each token:



3 Prefill Phase (Process the prompt once)

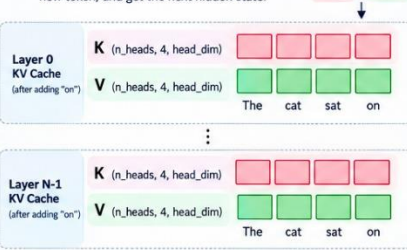
We process the prompt "The cat sat" and compute K and V for all prompt tokens, and store them in the cache (per layer).



4 Decode Step (Generate one new token)

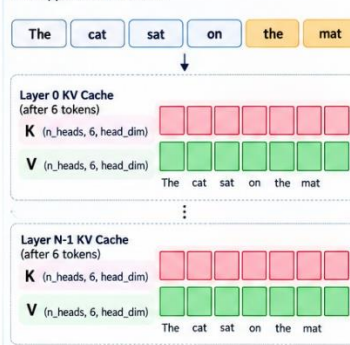
Let's generate the next token: "on" (as in "The cat sat on ...").

- 1 Compute K and V for the new token "on" (at each layer, only for this token).
- 2 Append (concatenate) the new K and V to the existing cache along the sequence length dimension.
- 3 Use the new token's Q to attend over ALL cached K (which now includes the new token) and get the next hidden state.



5 Repeat for Next Tokens

Next we generate "the", then "mat", and so on. Each time, we compute K/V for only the new token and append to the cache.



6 The Most Crucial Part: Appending to the Cache

This is the key operation in the code: we concatenate the new K and V to the existing ones along the sequence length axis (axis=1).



KV cache becomes much easier to understand once we separate three things:

1. Decoder blocks
2. Attention heads inside each block
3. Prefill vs. decode

The key idea is simple:

Cache the K and V from previous tokens so they do not have to be recomputed during generation.

1. GPT-2 has 12 decoder blocks

GPT-2 Small has 12 Transformer decoder blocks.

Each block contains self-attention and an MLP.

Inside each block, attention has 12 heads, with:

$$12 \text{ heads} \times 64 \text{ dimensions} = 768 \text{ hidden dimensions}$$

So:

$$\frac{768}{12} = 64$$

2. Token and position embeddings happen only once

Suppose our prompt is:

The deployment failed because

After tokenization:

T1 T2 T3 T4

We create the initial representation:

$$\mathbf{x} = \text{token_embedding} + \text{position_embedding}$$

giving:

$$X_0 = (4, 768)$$

This embedding step happens only once.

We do not add token + position embeddings again in Block 2, Block 3, etc.

3. Each block transforms the representation

Block 1 receives X_0 :

$$X_0 \rightarrow \text{LayerNorm} \rightarrow \text{Self-Attention} \rightarrow \text{MLP} \rightarrow X_1$$

Then Block 2 receives X_1 :

$$X_1 \rightarrow \text{LayerNorm} \rightarrow \text{Self-Attention} \rightarrow \text{MLP} \rightarrow X_2$$

So:

$$X_0 \rightarrow \text{Block 0} \rightarrow X_1$$

$$X_1 \rightarrow \text{Block 1} \rightarrow X_2$$

$$X_2 \rightarrow \text{Block 2} \rightarrow X_3$$

...

The layers are sequentially dependent, while the operations inside a layer are heavily parallelized.

4. Where do K and V come from?

Inside each block:

$$Q = XW_Q$$

$$K = XW_K$$

$$V = XW_V$$

So Block 0 might produce:

$$X_0 \rightarrow Q_0, K_0, V_0$$

while Block 1 produces:

$$X_1 \rightarrow Q_1, K_1, V_1$$

Therefore:

$$K_0 \neq K_1$$

$$V_0 \neq V_1$$

Each block has its own K/V because:

- The input representation is different
- The block has different $Q/K/V$ weights

So the cache contains separate K/V for every block.

5. Why do we cache K and V, but not Q?

Suppose we've already generated:

T1 T2 T3 T4

and now we're generating T_5 .
For the new token:

$$T_5 \rightarrow Q_5, K_5, V_5$$

We need Q_5 immediately:

$$Q_5 \times [K_1 \ K_2 \ K_3 \ K_4 \ K_5]^T$$

But after T_5 has been processed, we don't need Q_5 again.
Future tokens will generate their own queries:

$$T_6 \rightarrow Q_6$$

$$T_7 \rightarrow Q_7$$

$$T_8 \rightarrow Q_8$$

But they still need to look at T_5 .
Therefore:

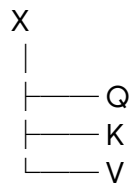
- $Q \rightarrow$ use temporarily
- $K \rightarrow$ cache
- $V \rightarrow$ cache

6. Prefill: process the prompt all at once

Suppose the prompt contains:

$T_1 \quad T_2 \quad T_3 \quad T_4$

During prefill, the whole prompt is processed.
For each block:



For example:

$$K = (12, 4, 64)$$

$$V = (12, 4, 64)$$

These are stored in the cache.

After prefill, the cache contains K/V histories for all 12 blocks, covering all prompt tokens.

7. Decode: process one new token

Now suppose the model predicts T_5 .

We start with:

$$T_5 \rightarrow \text{Token embedding} + \text{Position embedding} \rightarrow X$$

This X passes through Block 0.

The block computes:

$$Q_{05}, K_{05}, V_{05}$$

K_{05} and V_{05} are appended to Block 0's cache.

So:

- **Before:**

$$K_0 = [T1 \quad T2 \quad T3 \quad T4]$$

- **After:**

$$K_0 = [T1 \quad T2 \quad T3 \quad T4 \quad T5]$$

The same happens in Block 1, Block 2, and so on.

Each block updates its own cache.

8. The critical difference: no cache vs cache

Without KV cache

For a sequence of n tokens:

$$Q = (12, n, 64)$$

$$K = (12, n, 64)$$

Attention computes:

$$(12, n, 64) \times (12, 64, n) \rightarrow (12, n, n)$$

Every token gets a query, and every query compares against every key.

With KV cache during decode

Only the new token produces a query:

$$Q_{\text{new}} = (12, 1, 64)$$

while the cache contains:

$$K_{\text{full}} = (12, n, 64)$$

So:

$$(12, 1, 64) \times (12, 64, n) \rightarrow (12, 1, n)$$

The new token asks:

“How much attention should I give to each token in my history?”

The old K/V do not need to be recomputed.

9. The code reflects this directly

Your cached attention does:

`qkv = x @ c_attn_w + c_attn_b`

```
Q_new, K_new, V_new = np.split(
    qkv, 3, axis=-1
)
```

Then:

```
kv_cache.update(
    layer_idx,
    K_new,
    V_new
)
```

Only K and V are stored.

Then:

```

K_full, V_full = kv_cache.get(layer_idx)
scores = (
    Q_new
    @ K_full.transpose(0, 2, 1)
)

```

So the pattern is:

New token

```

|
|—— Q → use now
|—— K → append to cache
|—— V → append to cache

```

The most important distinction is:

Hidden states flow from one decoder block to the next. KV caches do not. Each block keeps its own K/V history.

And the most important decode rule is:

For every new token, calculate a fresh Q , K , and V at each block; cache the new K/V , but use Q only for the current attention calculation.